

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	6
1.2 Описание выходных данных.....	6
2 МЕТОД РЕШЕНИЯ.....	7
3 ОПИСАНИЕ АЛГОРИТМОВ.....	9
3.1 Алгоритм конструктора класса Device.....	9
3.2 Алгоритм деструктора класса Device.....	9
3.3 Алгоритм метода show класса Device.....	10
3.4 Алгоритм метода showStats класса Device.....	10
3.5 Алгоритм функции main.....	10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	12
5 КОД ПРОГРАММЫ.....	14
5.1 Файл Device.cpp.....	14
5.2 Файл Device.h.....	14
5.3 Файл main.cpp.....	15
6 ТЕСТИРОВАНИЕ.....	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	18

1 ПОСТАНОВКА ЗАДАЧИ

В информационной системе центра обработки данных ведётся учёт вычислительных устройств.

Каждое устройство имеет:

- имя устройства;
- уникальный номер устройства.

Кроме индивидуальных данных устройств, система хранит общую информацию:

- общее количество созданных устройств;
- текущее количество активных устройств.

Для реализации такой системы необходимо использовать статические поля класса.

Класс Устройство

Поля:

- строковое поле имени устройства;
- целочисленное поле уникального идентификатора устройства();
- общее количество созданных устройств;
- текущее количество активных устройств.

Методы:

- конструктор, сохраняющий имя устройства, увеличивающий счётчик созданных устройств, увеличивающий счётчик активных устройств, присваивающий уникальный номер устройства через счетчик созданных устройств.

- деструктор, уменьшающий число активных устройств;
- метод вывода данных объекта в формате "Device::show -> <name> <id>";

- статический метод вывода данных о созданных и активных объектах в формате "Device::showStats -> created: <value> active: <value>"

Алгоритм работы программы

1. Ввести три имени устройств.
2. Создать:
 - 2.1. один объект автоматически;
 - 2.2. два объекта динамически.
3. Для каждого объекта вывести установленные данные.
4. Вывести информацию о кол-ве созданных и активных устройств.
5. Удалить первый динамический объект.
6. Вывести информацию о кол-ве созданных и активных устройств.
7. Удалить второй динамический объект.
8. Завершить программу.

1.1 Описание входных данных

<name1> <name2> <name3>

1.2 Описание выходных данных

Device::show -> <name1> <id1>

Device::show -> <name2> <id2>

Device::show -> <name3> <id3>

Device::showStats -> created: <value> active: <value>

Device::showStats -> created: <value> active: <value>

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `device1` класса `Device` предназначен для хранения данных первого устройства, созданный автоматически;
- объект `device2` класса указатель на объект класса `Device` предназначен для хранения данных второго устройства, созданный динамически;
- объект `device3` класса указатель на объект класса `Device` предназначен для хранения данных третьего устройства, созданный динамически;
- функция `main` для организации работы программы;
- `static` для создания статических полей и методов.

Класс `Device`:

- свойства/поля:
 - поле хранение имени устройства:
 - наименование — `name`;
 - тип — `строка`;
 - модификатор доступа — `private`;
 - поле хранение уникального номера устройства:
 - наименование — `id`;
 - тип — `цел.`;
 - модификатор доступа — `private`;
 - поле хранение общего количества созданных устройств:
 - наименование — `totalCreated`;
 - тип — `цел.`;
 - модификатор доступа — `private`;

- о поле хранение текущего количества активных устройств:
 - наименование — `activeDevices`;
 - тип — цел.;
 - модификатор доступа — `private`;
- функционал:
 - о метод `Device` — параметризированный конструктор;
 - о метод `~Device` — деструктор;
 - о метод `show` — выводит данные объекта;
 - о метод `showStats` — выводит данные о созданных и активных объектах.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Device

Функционал: параметризованный конструктор.

Параметры: строк. name - имя устройства.

Алгоритм конструктора представлен в таблице 1.

Таблица 1 – Алгоритм конструктора класса Device

№	Предикат	Действия	№ перехода
1		присвоить полю name текущего объекта значение входного параметра name	2
2		инкрементировать статическое поле totalCreated	3
3		присвоить полю id значение поля totalCreated	4
4		инкрементировать статическое поле activeDevices	∅

3.2 Алгоритм деструктора класса Device

Функционал: деструктор.

Параметры: нет.

Алгоритм деструктора представлен в таблице 2.

Таблица 2 – Алгоритм деструктора класса Device

№	Предикат	Действия	№ перехода
1		декрементировать статическое поле activeDevices	∅

3.3 Алгоритм метода show класса Device

Функционал: выводит данные объекта.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода show класса Device

№	Предикат	Действия	№ перехода
1		вывод в консоль "Device::show -> ", name, " ", id, endl	Ø

3.4 Алгоритм метода showStats класса Device

Функционал: выводит данные о созданных и активных объектах.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода showStats класса Device

№	Предикат	Действия	№ перехода
1		вывод в консоль "Device::showStats -> created: ", totalCreated, " active: ", activeDevices, endl	Ø

3.5 Алгоритм функции main

Функционал: организация работы программы.

Параметры: нет.

Возвращаемое значение: цел. код возврата программы.

Алгоритм функции представлен в таблице 5.

Таблица 5 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявл. строк. перем. name1, name2 и name3	2
2		ввод значений в перем. name1, name2 и name3	3
3		создать объект device1 класса Device, передав в конструктор name1	4
4		динамически создать объект класса Device с аргументом name2 и присвоить его адрес указателю device2	5
5		динамически создать объект класса Device с аргументом name3 и присвоить его адрес указателю device3	6
6		вызов метода show у объекта device1	7
7		вызов метода show по указателю device2	8
8		вызов метода show по указателю device3	9
9		вызов статического метода showStats класса Device	10
10		удалить объект по указателю device2	11
11		вызов статического метода showStats класса Device	12
12		удалить объект по указателю device3	13
13		вернуть 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-2.

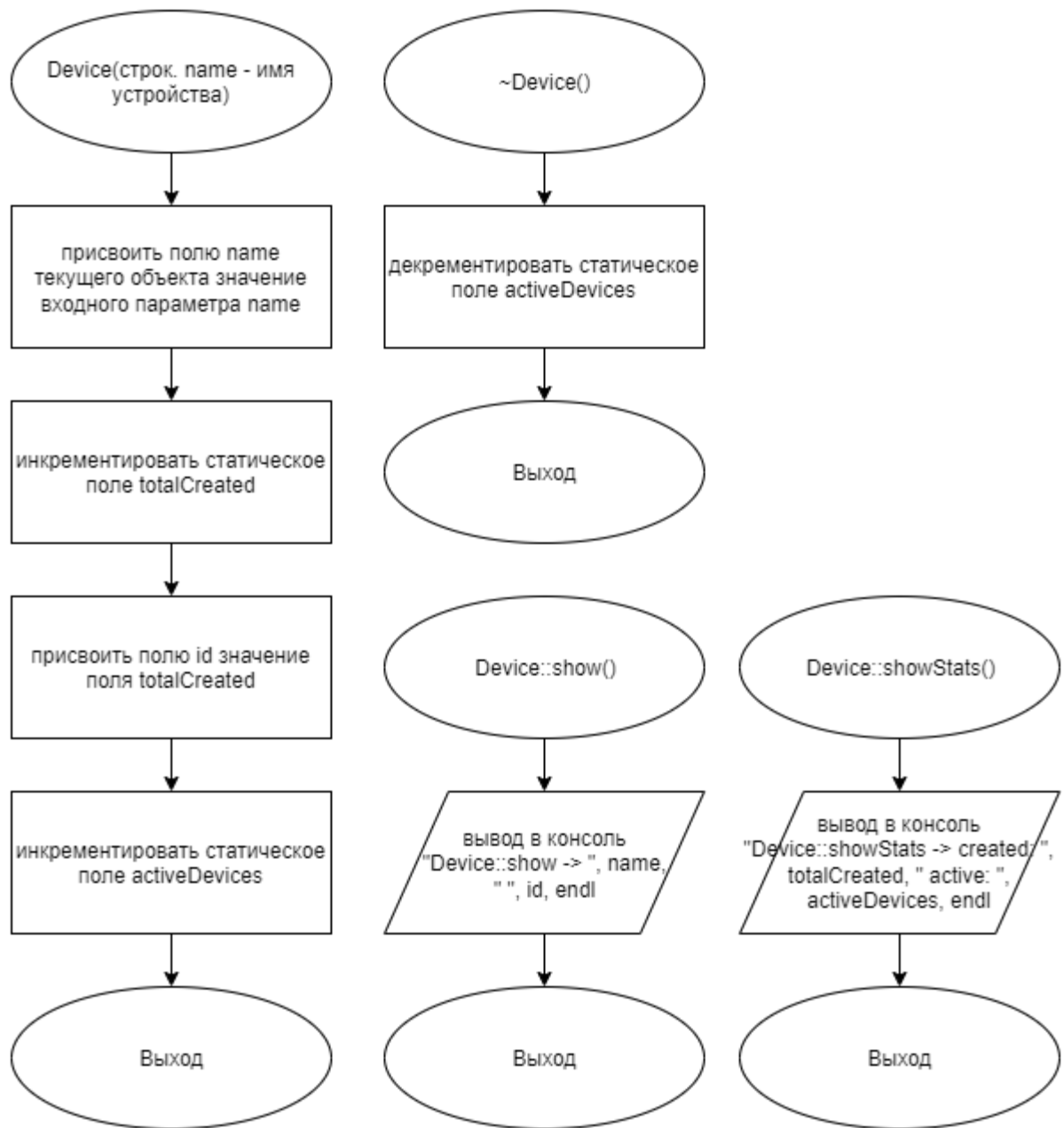


Рисунок 1 – Блок-схема алгоритма

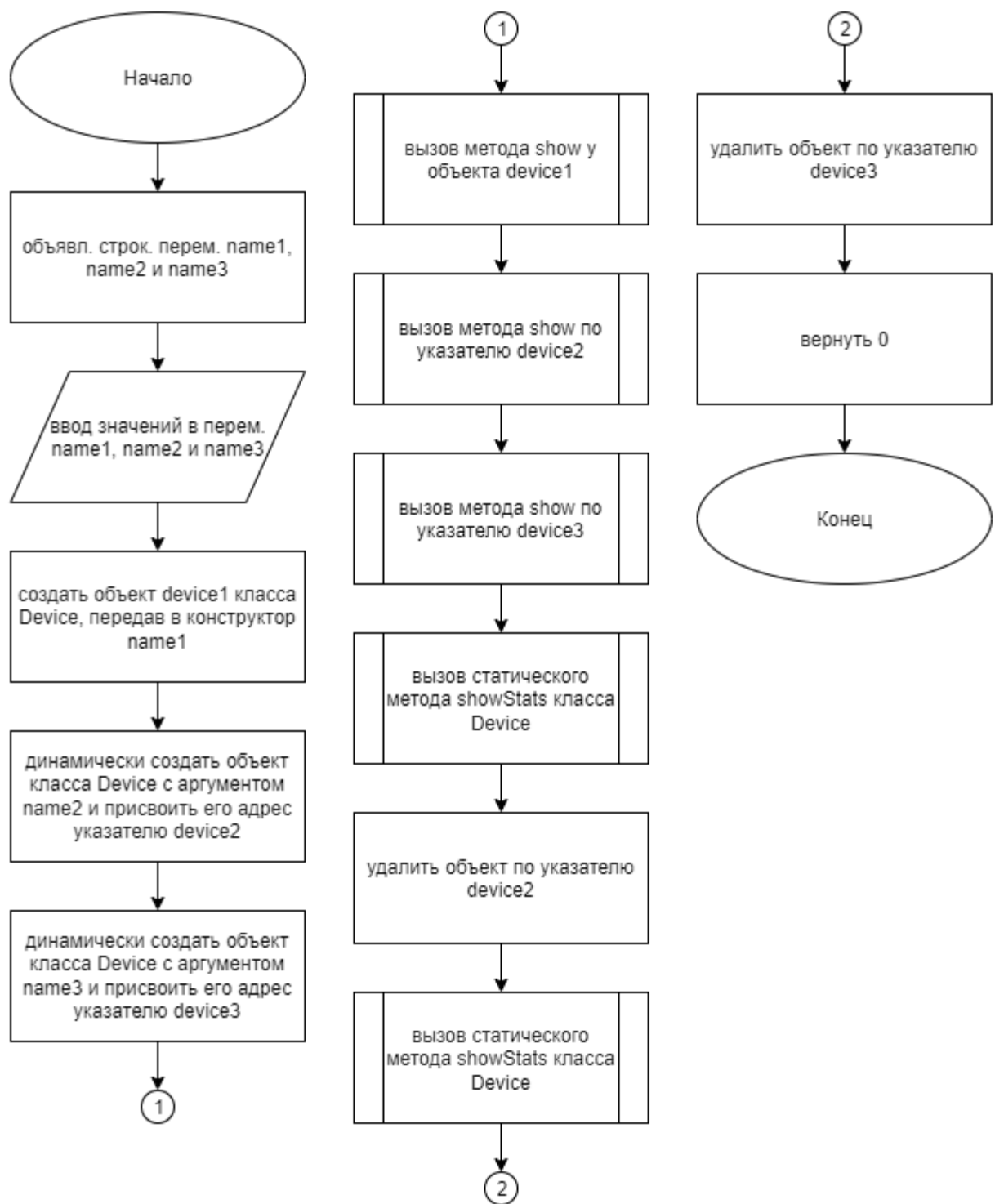


Рисунок 2 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Device.cpp

Листинг 1 – Device.cpp

```
#include "Device.h"

int Device::totalCreated = 0;
int Device::activeDevices = 0;

Device::Device(string name) {
    this->name = name;

    totalCreated++;
    id = totalCreated;
    activeDevices++;
}

Device::~Device() {
    activeDevices--;
}

void Device::show() {
    cout << "Device::show -> " << name << " " << id << endl;
}

void Device::showStats() {
    cout << "Device::showStats -> created: " << totalCreated
        << " active: " << activeDevices << endl;
}
```

5.2 Файл Device.h

Листинг 2 – Device.h

```
#ifndef __DEVICE__H
#define __DEVICE__H

#include <iostream>
```

```

#include <string>
using namespace std;

class Device {
private:
    std::string name;
    int id;
    static int totalCreated;
    static int activeDevices;

public:
    Device(string name);
    ~Device();

    void show();
    static void showStats();
};

#endif

```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>

#include "Device.h"
#include <string>
using namespace std;

int main() {
    string name1, name2, name3;

    cin >> name1 >> name2 >> name3;

    Device device1(name1);

    Device* device2 = new Device(name2);
    Device* device3 = new Device(name3);

    device1.show();
    device2->show();
    device3->show();

    Device::showStats();

    delete device2;

    Device::showStats();
}

```

```
    delete device3;  
    return 0;  
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 6.

Таблица 6 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Desktop Laptop Phone	Device::show -> Desktop 1 Device::show -> Laptop 2 Device::show -> Phone 3 Device::showStats -> created: 3 active: 3 Device::showStats -> created: 3 active: 2	Device::show -> Desktop 1 Device::show -> Laptop 2 Device::show -> Phone 3 Device::showStats -> created: 3 active: 3 Device::showStats -> created: 3 active: 2

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).